Data Structure

HW1

-Omok-

Report

20211562 이동협

Project subject

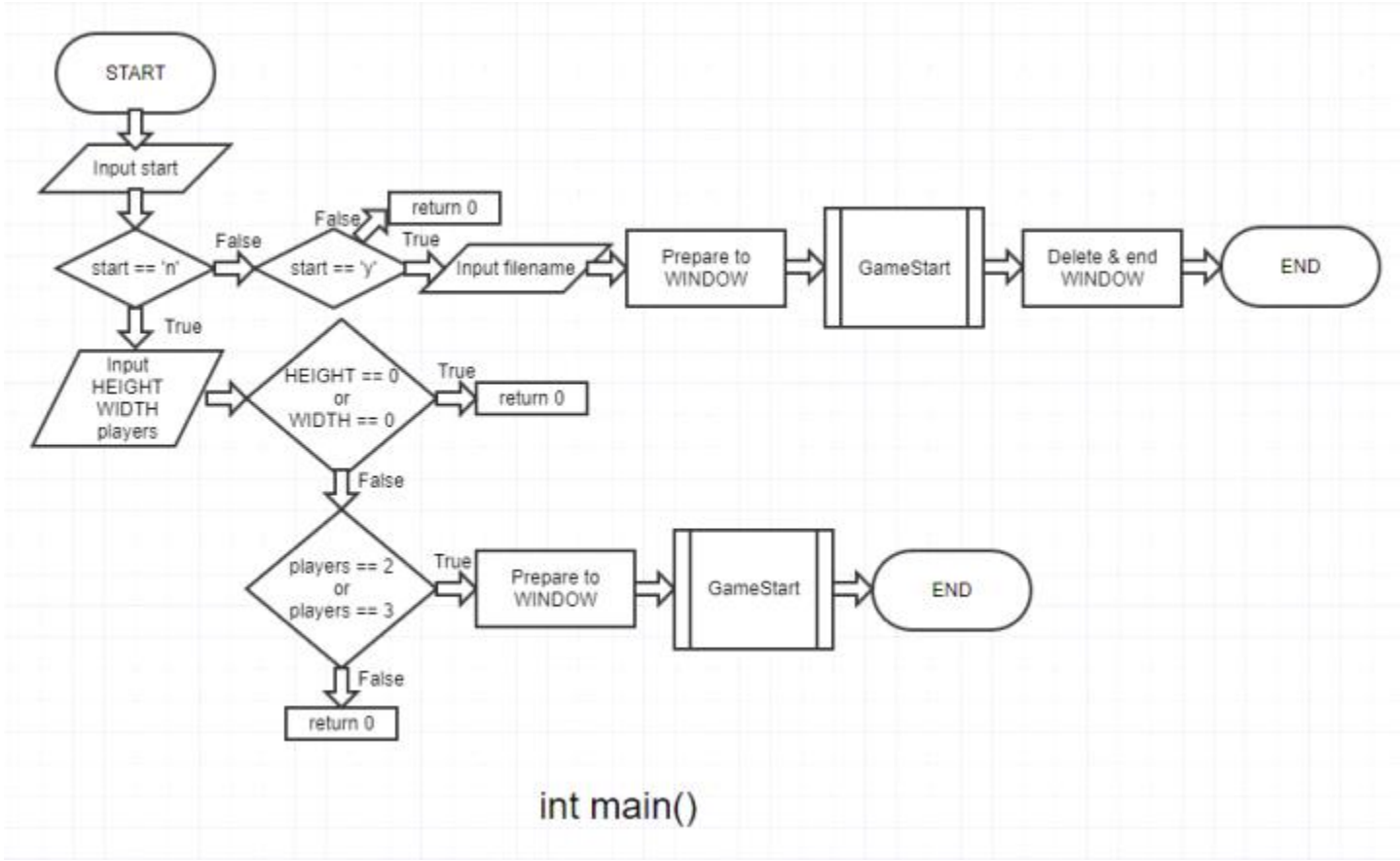
- Implement Omok game.

Goal of the project

- implement program of the omok game that satisfies below details.
  - User can play the omok game with 2 players or 3 players.
  - When 2p game, it follows the omok's rule.
  - When 3p game, it follows the samok's rule.
  - When start the program, get omokboard's size(HEIGHT, WIDTH) by input.
  - If player press arrow key, cursor moves by arrow key's direction.
  - If player press Enter or Space key, set the current player's stone on cursor's position.
  - After setting stone, check if there's winner. If there's winner, print who's win and end the game.
  - During the game, if player press '1' key, get the filename by input and save the game status(current cursor's position, omok board, turn, etc..) to the file.
  - During the game, if player press '1' key, end the game without save.

Flow chart of the project

1. int main()

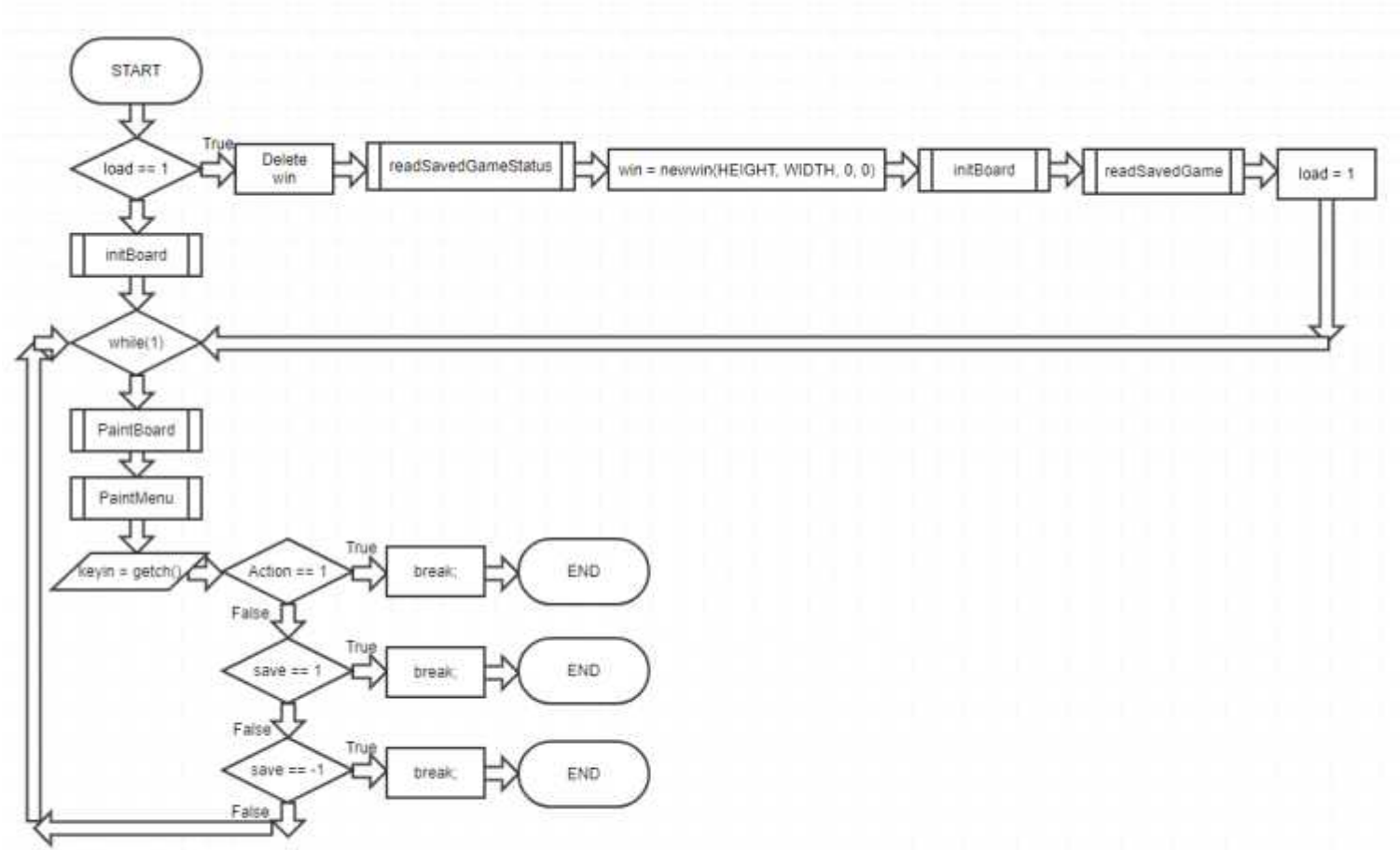


This function firstly check “load savefile or not”.

If user doesn’t load game, get input of gameboard’s size(HEIGHT, WIDTH) and player number(2or3). and goto GameStart() by given value.

Else if user load game, get input of savefile’s name. and goto GameStart() by given value. Lastly, when GameStart function ended, terminate the program.

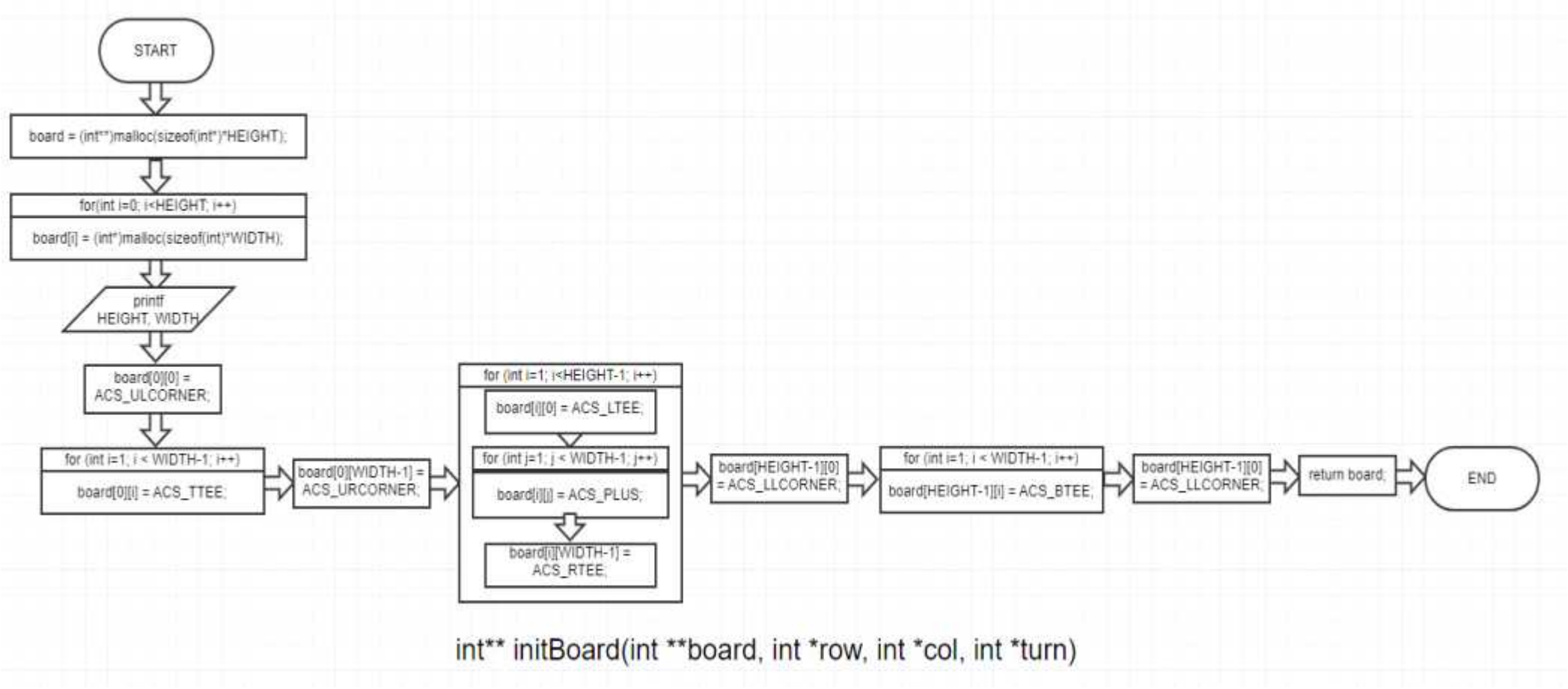
2. void gameStart(WINDOW *win, char* filename, int load, int players)



In this function, it firstly check “load game or not”. if load needed, read gamefile, re-declare window by given HEIGHT, WIDTH. and initialize omok-board by gamefile. Else if load isn’t in progress, intialize board by given HEIGHT, WIDTH. Then process goes to while loop.

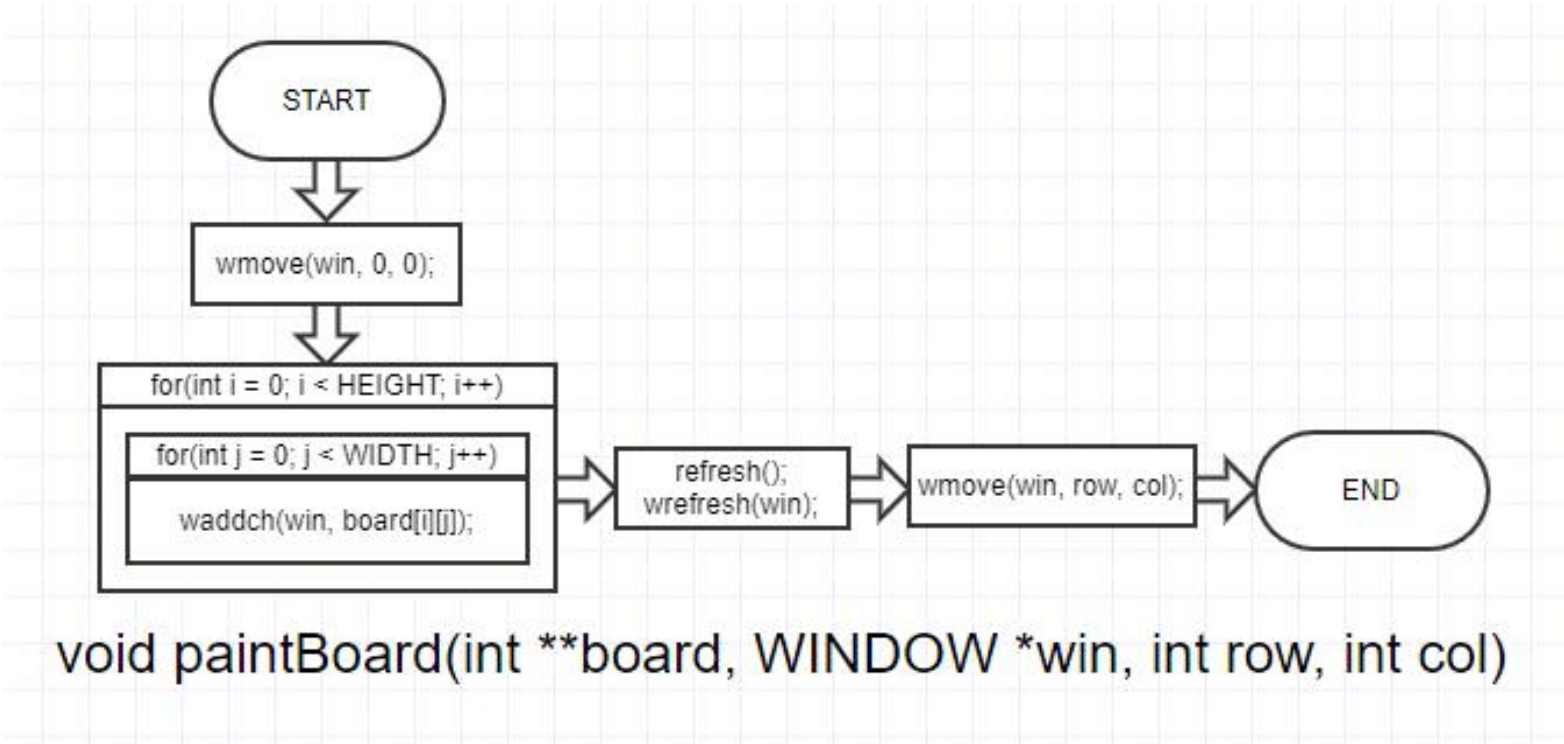
Next, paint the omok board and gamemenu. and wait for player’s keyboard type. And if there’s a winner or player saved this game or player exited the game, break the loop.

3. int** initBoard(int **board, int *row, int *col, int *turn)



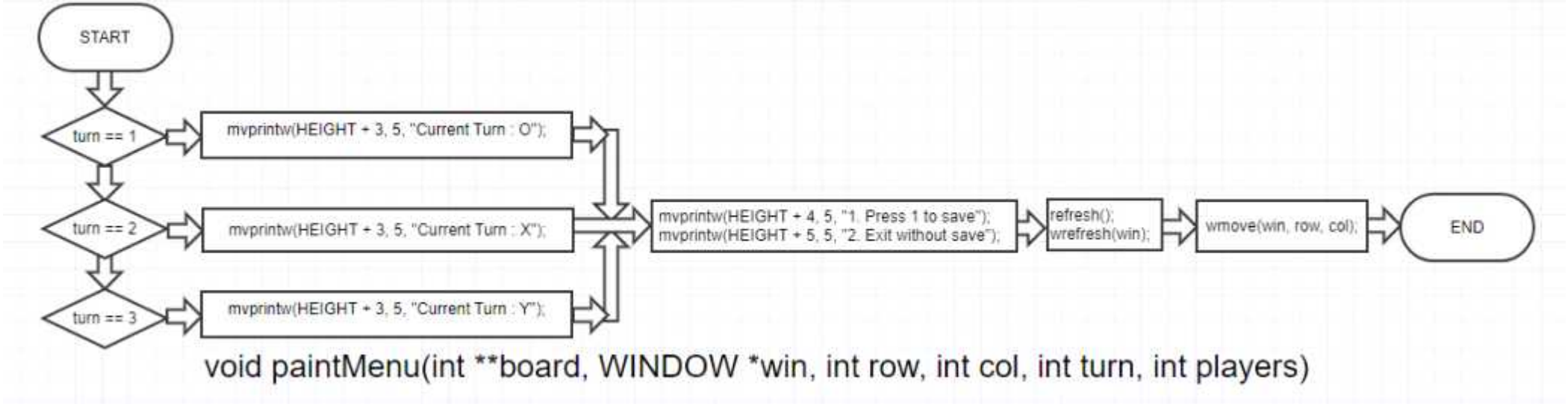
initBoard's main function is save OmokBoard's shape at "int** board(two-dimension pointer)".
It allocates memory to int** board by given HEIGHT, WIDTH size.

4. void paintBoard(int **board, WINDOW *win, int row, int col)



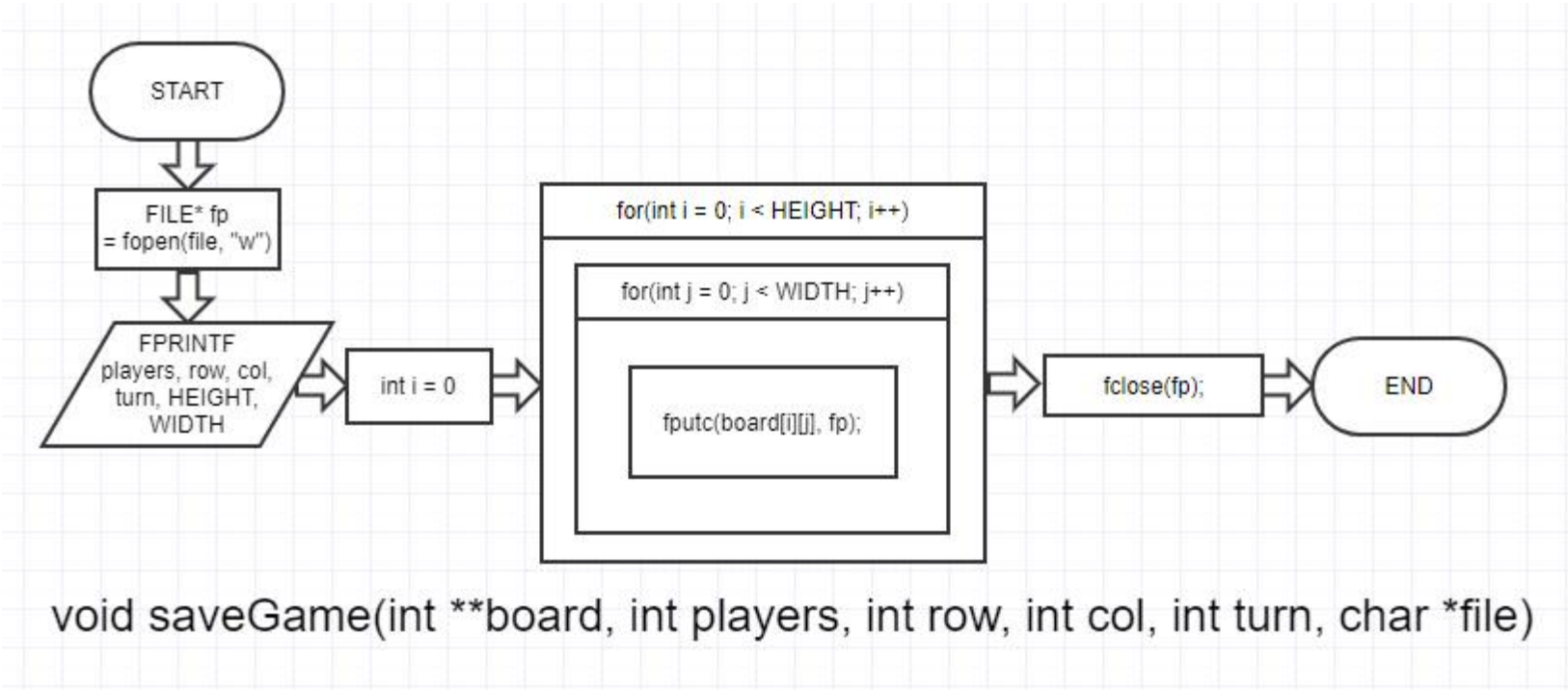
This function shows the omok board on the window.
It moves to (0, 0) position and set each cell on win by waddch().
And use refresh() to show win.
Lastly, move cursor to latest cursor position.

5. void paintMenu(int **board, WINDOW *win, int row, int col, int turn, int players)



It firstly shows current turn's player and show the how to save or exit the game.
And refresh the window, move cursor to latest position of cursor.

6. void saveGame(int **board, int players, int row, int col, int turn, char *file)



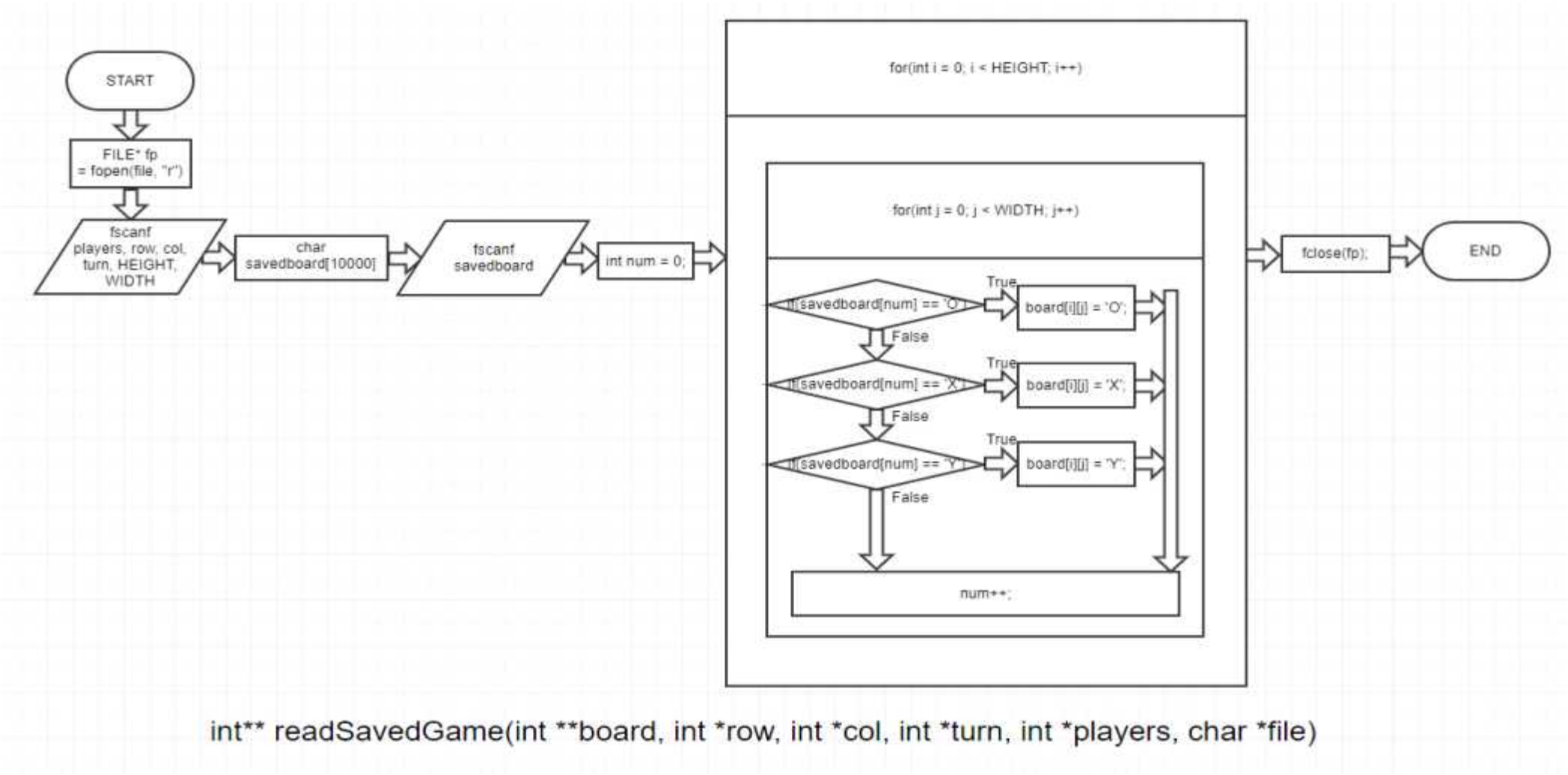
It firstly declare file pointer to save game.

Then it write playernumber, current row, current col, current turn, HEIGHT, WIDTH at the first row of file.

And at the second row of file, it writes gameboard's shape.

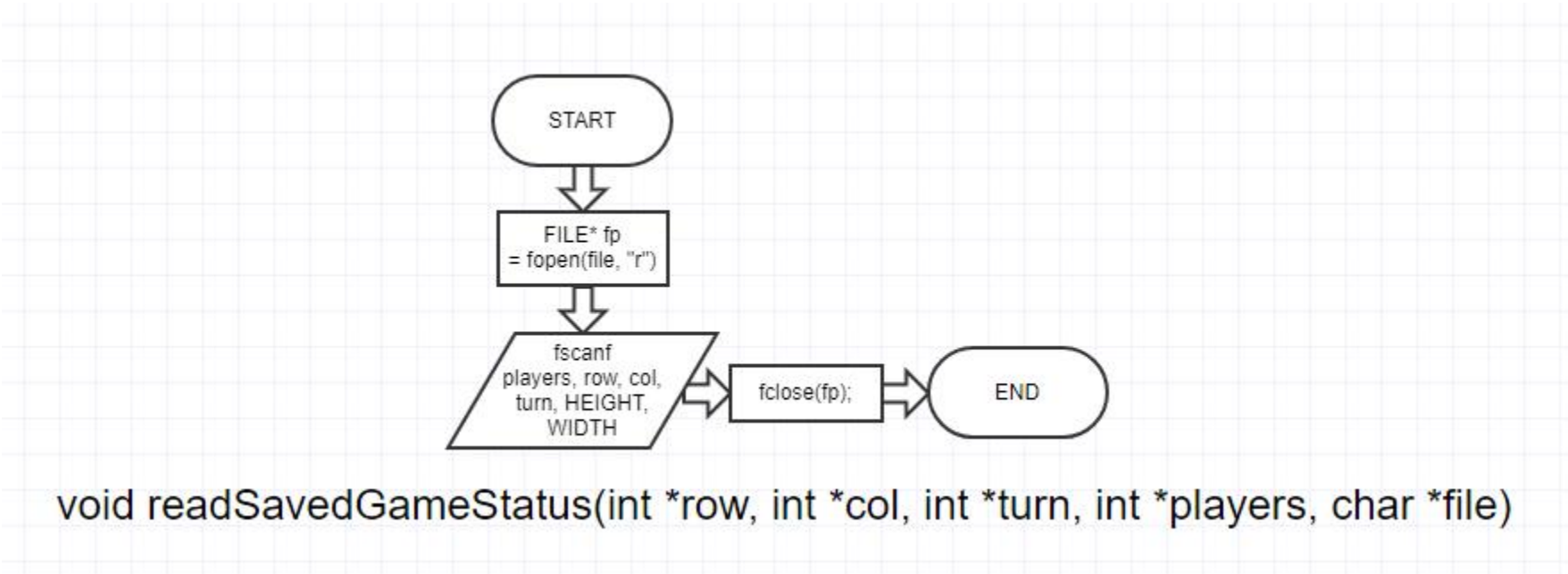
Lastly, close the file pointer and end the funciton.

7. int** readSavedGame(int **board, int *row, int *col, int *turn, int *players, char *file)



Firstly, it opens savefile and read playernumber, current row, current col, current turn, HEIGHT, and WIDTH at the first row of savefile.
And it reads savedBoard's shape at the second row of savefile.
Lastly, it closes the file pointer and end the function.

8. void readSavedGameStatus(int *row, int *col, int *turn, int *players, char *file)

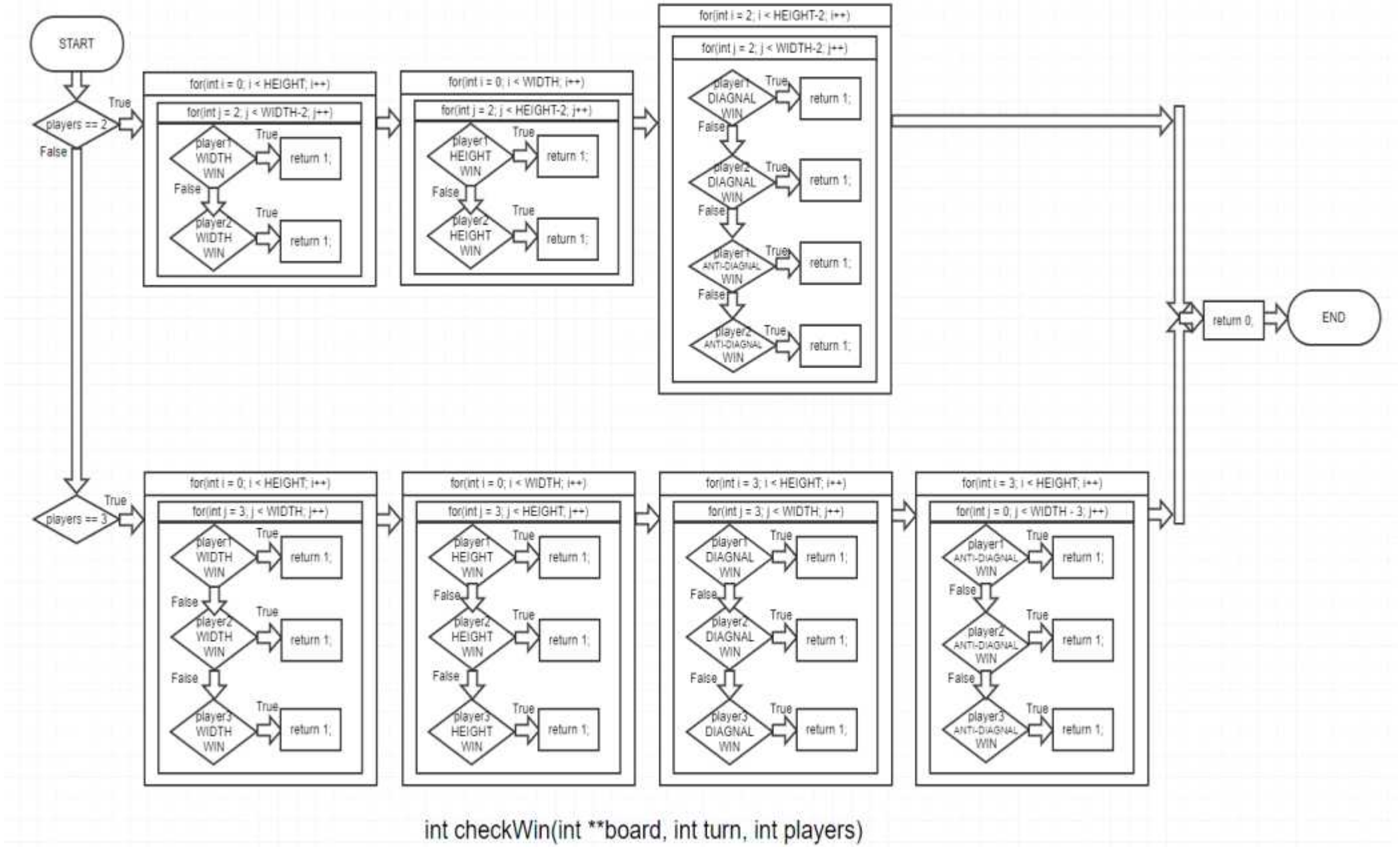


Actually, this function is brief version of readSavedGame.

But it is designed to [redeclare WINDOW* win by given HEIGHT, WIDTH] before operation of initBoard.

If this function doesn't exist, gameboard is loaded abnormally.

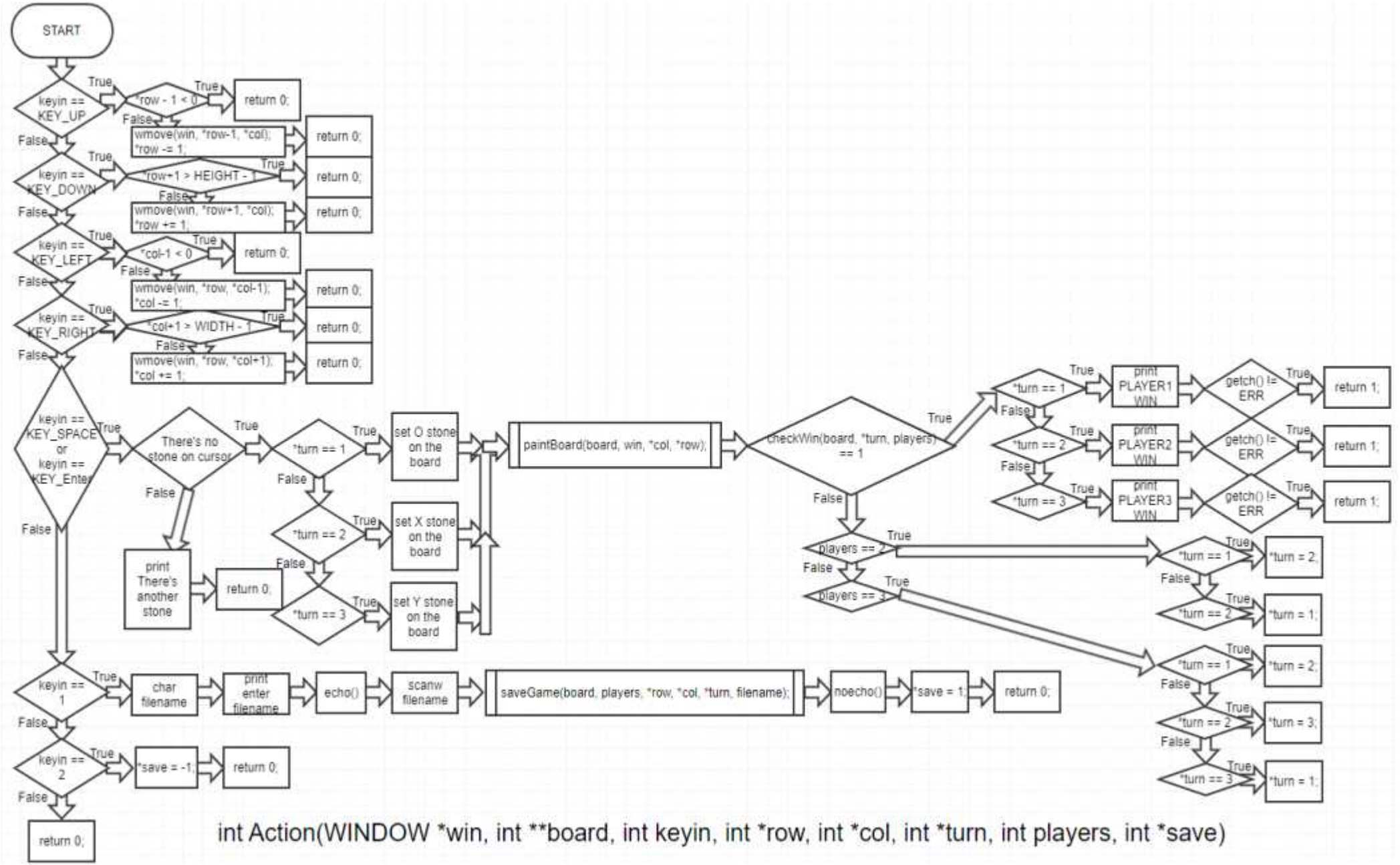
9. int checkWin(int **board, int turn, int players)



int checkWin(int **board, int turn, int players)

This function check the board and if there's winner, return 1.
Firstly, it check the player num.
If playernum is 2, it check player1&player2's HEIGHT WIN, WIDTH WIN, DIAGNAL&ANTI-DIAGNAL WIN by rule of omok.
If playernum is 3, it check player1&player2&player3's HEIGHT WIN, WIDTH WIN, DIAGNAL&ANTI-DIAGNAL WIN by rule of samok.
If there's winner, it returns 1. Or it returns 0.

10. int Action(WINDOW *win, int **board, int keyin, int *row, int *col, int *turn, int players, int *save)

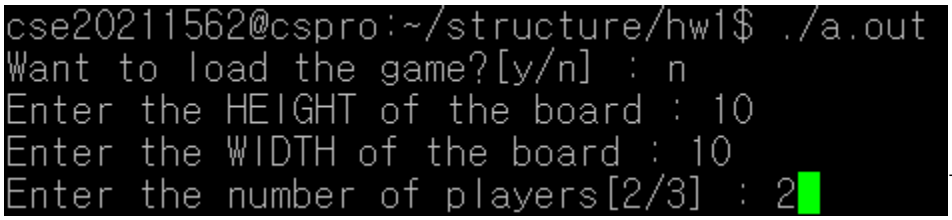


Firstly, it checks which key is player lastly typed.

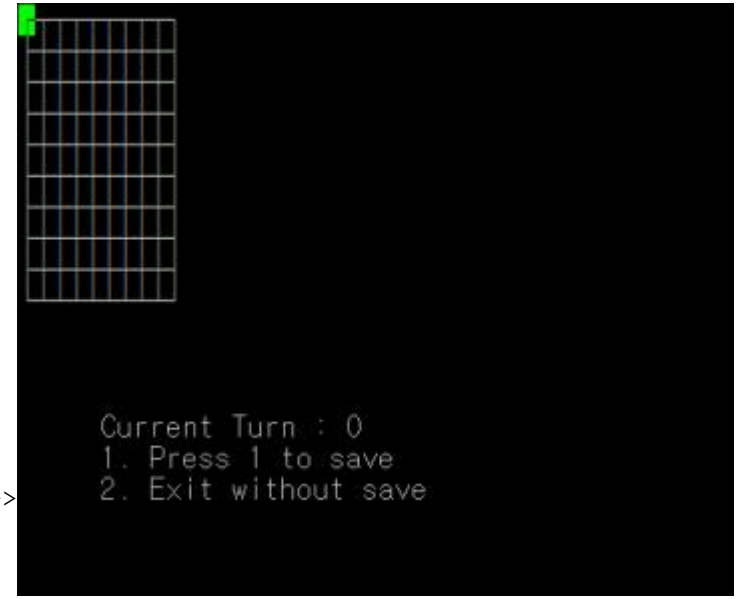
1. lastly typed key is “Arrow key(→←↑↓)”: check that movement doesn’t move out of board. It doesn’t move out the board, it moves cursor and change turn to next player’s turn.
2. Lastly typed key is “Enter or Space key”: check if at cursor’s position, there’s stone already or not.
   There’s no stone, set current player’s stone and change turn to next player’s turn.
   There’s stone, print error messege and end the function.
3. Lastly typed key is “1” : get input of savefile name(when player types the filename, that typing is printed on window) and call saveGame to save game. And set int* save 1 to end the program at gameStart.
4. Lastly typed key is “2” : It ends the game without save. set int* save -1 to end the program at gameStart.
5. Lastly typed key is none of 1.2.3.4. : return 0 to end the function.

Test of the project

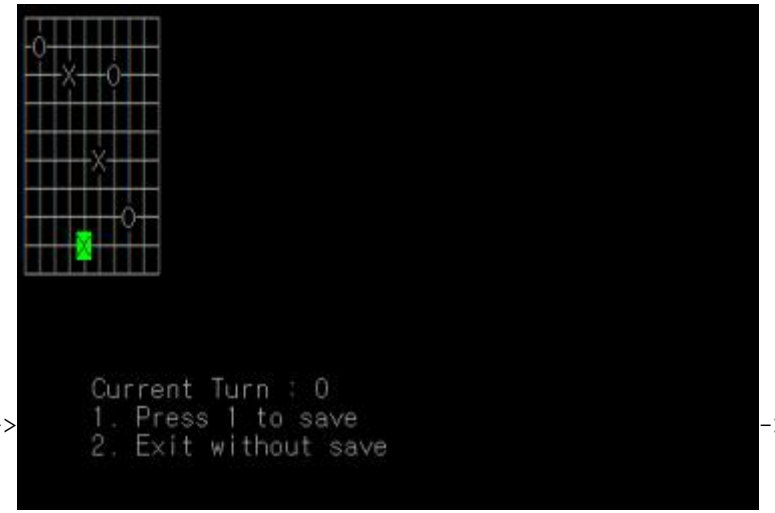
1. load : n, Board : 10x10, Player : 2. HEIGHT Win



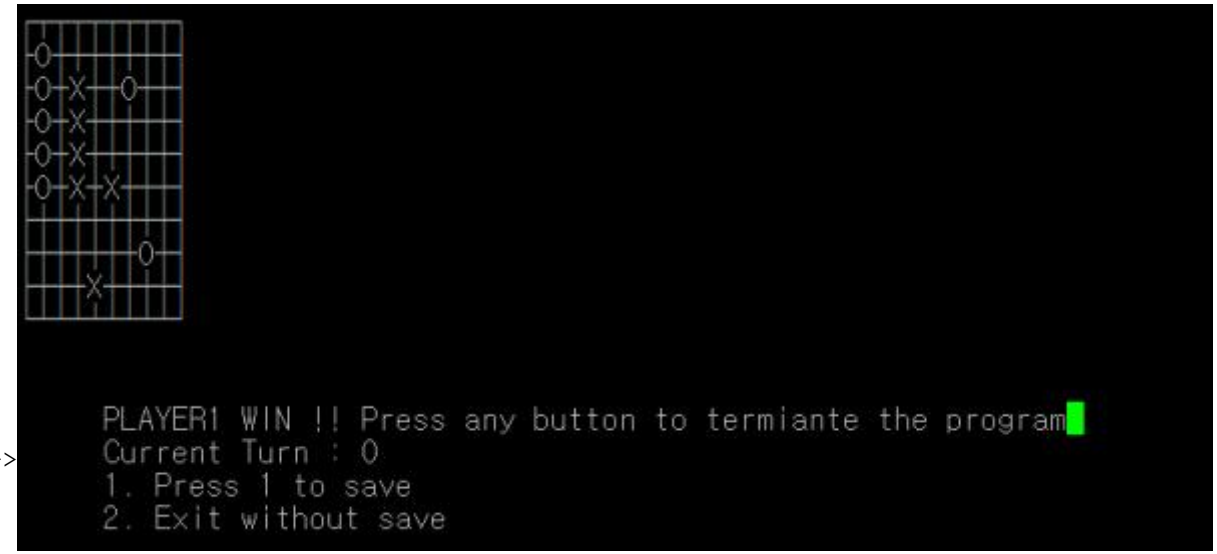
->



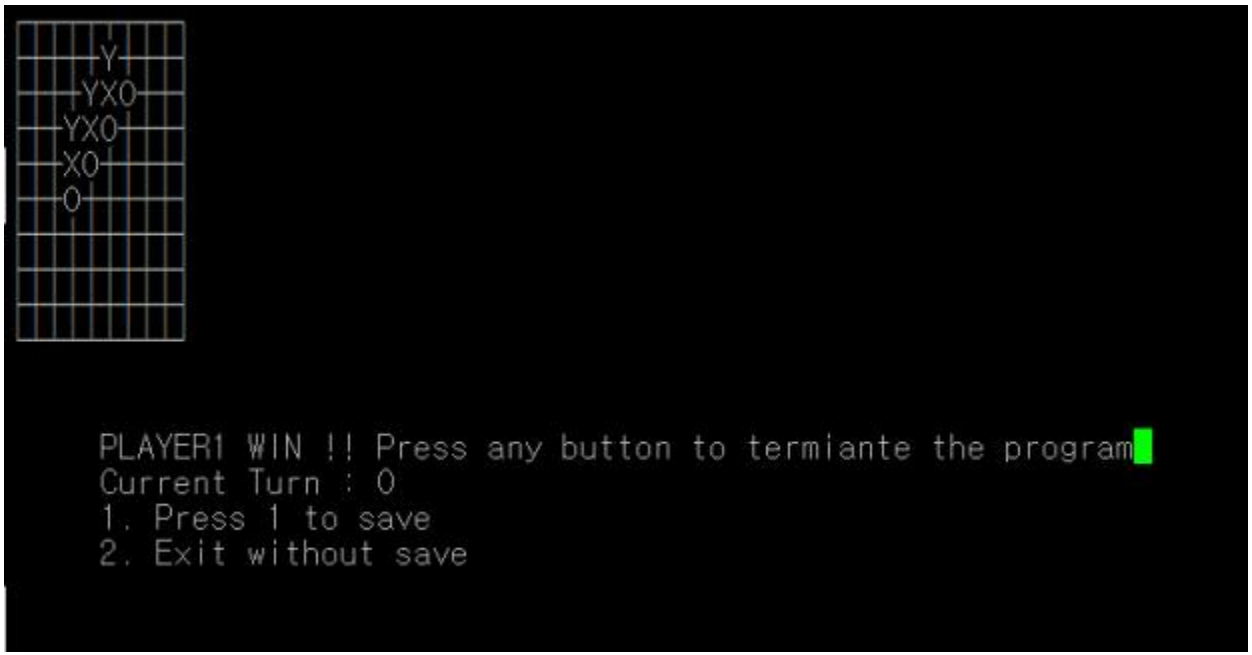
->



->



2. load : n, Board : 10x10, Player : 3. ANTI-DIAGNAL Win



3. load : n, Board : 7x7, player : 2 -> savefile(save.txt) -> load : y, save.txt

->

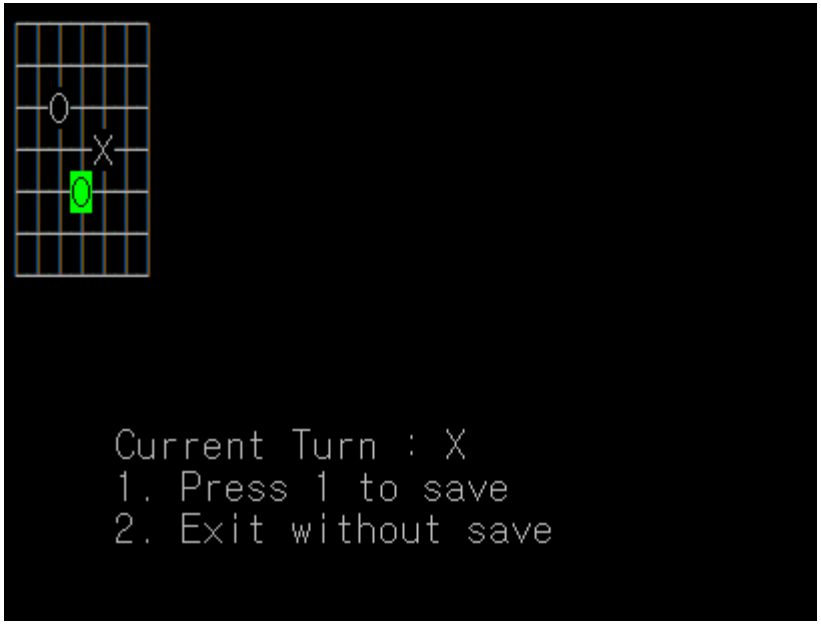
->

```
cse20211562@cspro:~/structure/hw1$ ./a.out
Want to load the game?[y/n] : n
Enter the HEIGHT of the board : 7
Enter the WIDTH of the board : 7
Enter the number of players[2/3] : 2
Game Ended.
cse20211562@cspro:~/structure/hw1$
```

->

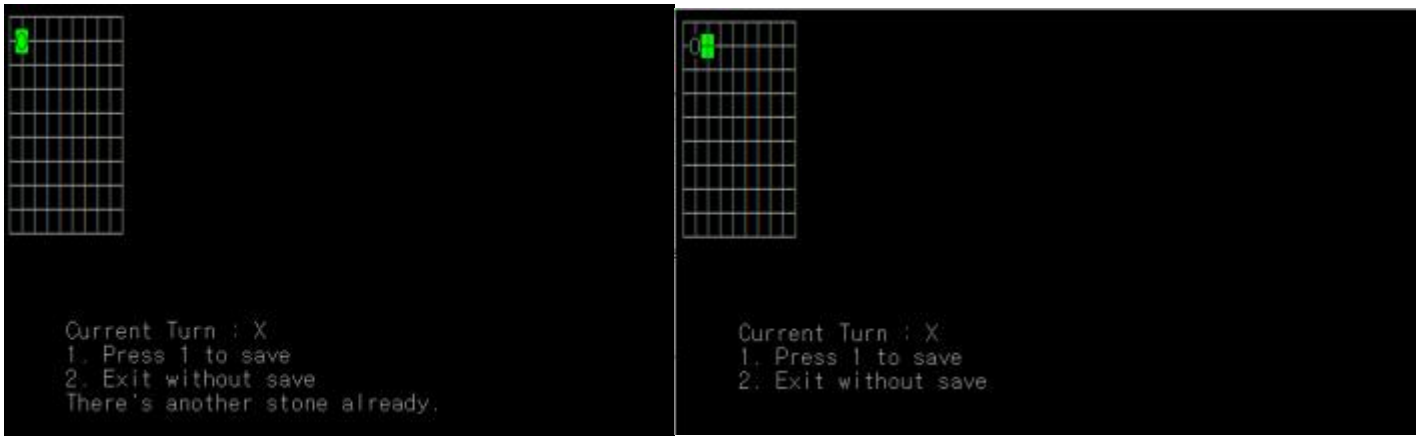
```
cse20211562@cspro:~/structure/hw1$ ./a.out
Want to load the game?[y/n] : y
Enter the name of the file : save.txt
```

->



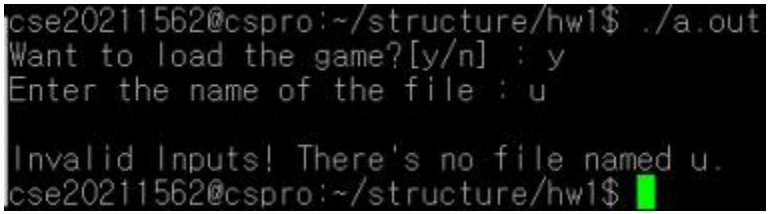
ERROR CASE

1. Already set stone



When player tries to set stone on already set stone, it prints “There’s another stone already. And player moves the cursor, it disappears.

2. Invalid input



When scan invalid input, it prints error message and terminate the program.

3. Cursor movement limit

It doesn't move out of the limit of the board.